Registros y tipos abstractos de datos

Estructura, diseño y práctica aplicada
Python

¿Qué es un registro?

Definición

Un registro(record) es una estructura de datos que agrupa **campos relacionados** bajo una misma entidad lógica.

Composición

A diferencia de una lista simple, un registro combina múltiples tipos de datos (enteros, cadenas, booleanos) para representar un objeto coherente.

Ejemplos cotidianos de registros

Estructuras de datos que agrupan información lógica del mundo real.

Persona
PERFIL

- nombre
- edad
- DNI
- correo

Libro
BIBLIOGRAFÍA

- título
- autor
- ISBN
- año

Producto
INVENTARIO

- id
- precio
- stock
- categoría

Punto 2D
GEOMETRÍA

- x
- y
- *Coordenadas cartesianas*

Evento
AGENDA

- fecha
- lugar
- asistentes

Implementación Básica
Registros conDiccionarios

Implementación Básica

¿Cuándo usarlos?

Ideales para datos **ad-hoc** y flexibles, cuando la estructura no es fija o proviene de fuentes externas como JSON. No requieren definición de clase previa.

Ventajas

- ✓ Muy flexibles y dinámicos
- ✓ Sintaxis literal sencilla
- ✓ Compatibles con JSON.

Desventajas

- ⊗ Sin validación de datos
- ⊗ Sin autocompletado
- ⊗ Uso de memoria alto.

registro_dict.py

```python
# 1. Definición ad-hoc (sin clase previa)
persona = {
    "nombre": "Ana",
    "edad": 21,
    "activo": True
}

# 2. Agregar campos dinámicamente
persona["email"] = "ana@example.com"

# 3. Acceso seguro (evita KeyError)
# Si 'edad' no existe, devuelve 0
edad_actual = persona.get("edad", 0)

# 4. Iteración sobre pares clave-valor
print("--- Datos del Registro ---")
for campo, valor in persona.items():
    print(f"{campo}: {valor}")

# Resultado:
# nombre: Ana
# edad: 21
# ...
```

Registros con NamedTuples

Estructuras Inmutables

¿Cuándo usarlas?

Perfectas para estructuras de datos ligeras e inmutables. Ofrecen legibilidad mediante acceso por atributos sin sobrecarga de una clase completa.

Ventajas

- ✓ Muy eficientes en memoria
- ✓ Hashables (claves de dict)
- ✓ Acceso legible (punto)

Desventajas

- ⊗ Validación limitada
- ⊗ API fija (difícil de extender)
- ⊗ Inmutables (copia total)

registro_namedtuple.py

```python
from collections import namedtuple

# 1. Definición de la estructura (nombre tipo, campos)
Estudiante = namedtuple("Estudiante", "nombre edad activo")

# 2. Creación de una instancia
e1 = Estudiante("Ana", 21, True)

# 3. Acceso legible por atributo
print(f"Estudiante: {e1.nombre}, Edad: {e1.edad}")
# Salida: Estudiante: Ana, Edad: 21

# 4. Modificación (Crear copia modificada)
# e1.edad = 22  # Error: AttributeError (es inmutable)

# Usamos _replace para crear una nueva instancia
e2 = e1._replace(edad=22)

print(f"Original: {e1.edad}, Nuevo: {e2.edad}")
# Salida: Original: 21, Nuevo: 22
```

registro_dataclass.py

```python
from dataclasses import dataclass

@dataclass
class Estudiante:
    # Definición clara de tipos y valores por defecto
    nombre: str
    edad: int
    promedio: float = 0.0
    activo: bool = True

    def __post_init__(self):
        # Validación automática tras la creación
        if self.edad < 0:
            raise ValueError("La edad no puede ser negativa")

    def es_mayor(self) -> bool:
        # Método personalizado de negocio
        return self.edad >= 18

# Uso del registro
try:
    e = Estudiante("Ana", 21, 8.5)
    print(e)            # __repr__ automático y legible
    print(e.es_mayor()) # True

    # Esto lanzaría error por la validación:
    e_error = Estudiante("Luis", -5)
except ValueError as err:
    print(f"Error: {err}")
```

Registros con Dataclasses

Modelos Robustos (Python 3.7+)

La opción moderna

Ideales para definir modelos de datos claros con **tipado estático** (hints), valores por defecto y lógica de negocio. Generan automáticamente código repetitivo como _init_ y _repr_.

Ventajas

- ✓ Código conciso y legible.
- ✓ Validación post-init.
- ✓ Soporte de Types Hints.

Desventajas

- ⊗ Requiere Python 3,7+
- ⊗ Ligeramente más pesado.

Análisis Técnico
Comparación de Estructuras

Criterio	Dictionary Dict	NamedTuple Collections.namedTuple	DataClass @dataclass
Mutabilidad	Mutable	Inmutable	Mutable (opcional)
Validación	Manual	Dificil (Override)	Nativa (_post_int_)
Tipado/Hints	No (Generico)	Limitado	Excelente
Memoria/CPU	Pesado	Muy ligero	Medio
Representación	{‘a’:a}	T(a=1)	C(a=1)

MEJOR PARA...

Datos **ad-hoc**, JSON, configs rápidas o cuando los nombres de los campos no se conocen hasta el runtime.

MEJOR PARA...

Estructuras **simples e inmutables**. Coordenadas (x,y), resultados de DB, o retornos de funciones. Rendimiento crítico.

MEJOR PARA...

Modelado de dominio. Objetos con lógica, validación, métodos propios y tipos definidos. El estándar moderno.

Fundamentos de Diseño

Tipos Abstractos de Datos (ADT)

Definición

Un ADT especifica un conjunto de valores y operaciones (comportamientos), sin fijar como se implementa internamente.

Idea Central

Define una interfaz publica estable. Esto permite cambiar la implementación interna (optimización) sin afectar al código que usa el dato.

Principios de Diseño

Encapsulación y Abstracción

Encapsulación

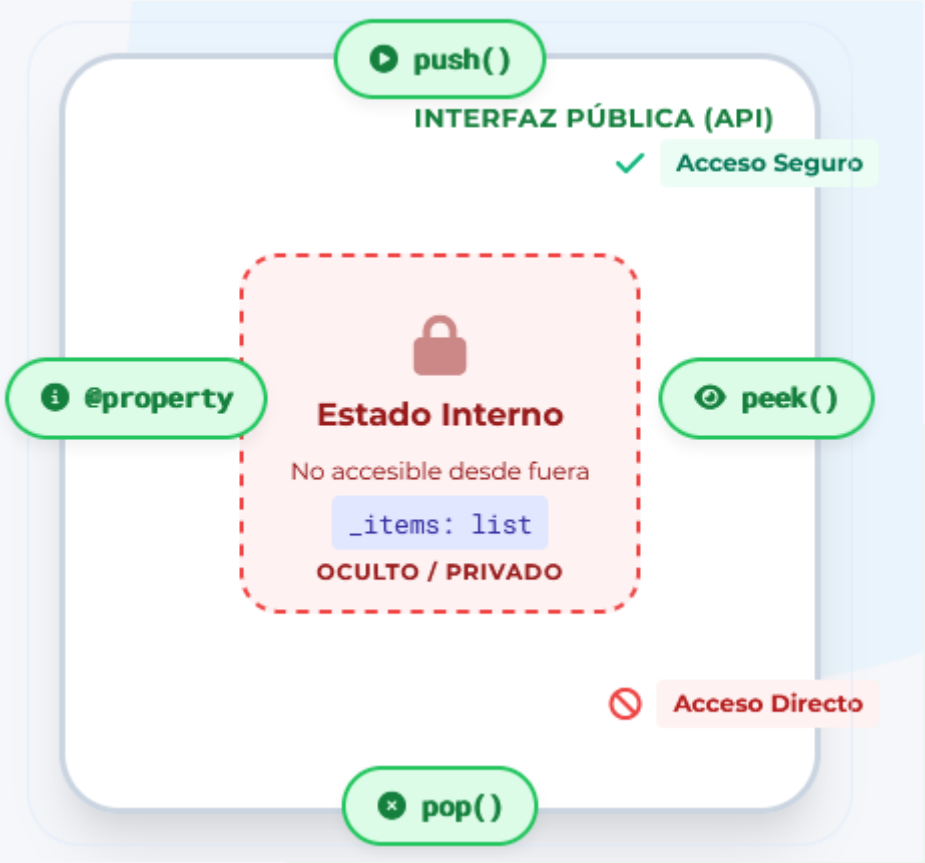
Oculta el estado interno y protege la integridad de los datos. El acceso directo a los datos se restringe, permite solo operaciones seguras a través de la interfaz.

Abstracción

Define QUÉ hace el tipo de dato, ignorando el CÓMO. Permite cambiar la implementación interna (código) sin romper el código que utiliza.

En Python

Uso de convención _variable para “privado” y decoradores @property para getters/setters controlados.



Beneficios Clave

Ventajas de usar ADT

Por qué encapsular datos y operaciones mejora drásticamente la calidad del software.

Mantenibilidad

Permite cambiar la implementación interna (el CÓMO) sin romper la interfaz pública ni afectar al código cliente.

1

Reutilización

Facilita la creación de componentes genéricos y modulares que pueden aplicarse a múltiples problemas.

2

Seguridad

Garantiza el cumplimiento de invariantes, impidiendo que el objeto alcance estados inválidos o corruptos.

3

Testeo

Permite aislar componentes para verificar su comportamiento esperado de forma independiente y robusta.

4

Rendimiento

Posibilita la elección y optimización de estructuras de datos internas según las necesidades de velocidad.

5

contador_adt.py

```python
class Contador:
    """Un contador simple que nunca es negativo."""

    def __init__(self, valor_inicial: int = 0):
        # Validación inicial (Invariante)
        if valor_inicial < 0:
            raise ValueError("No puede ser negativo")
        # Estado interno 'privado' (convención _)
        self._valor = valor_inicial

    @property
    def valor(self) -> int:
        """Acceso público de solo lectura."""
        return self._valor

    def inc(self, paso: int = 1) -> None:
        """Modifica el estado de forma controlada."""
        if paso <= 0:
            raise ValueError("El paso debe ser positivo")
        self._valor += paso

    def reset(self) -> None:
        """Reinicia el contador a cero."""
        self._valor = 0

    def __repr__(self) -> str:
        """Representación para debugging."""
        return f"Contador(valor={self._valor})"

# Uso del ADT
c = Contador(10)
c.inc()      # Estado interno cambia a 11
print(c.valor) # Acceso seguro: 11
# c.valor = 5  # Error: AttributeError (propiedad sin setter)
```

Estructura y Encapsulación

Implementación de ADT con Clases

Clases como contenedores

En Python, las clases son el mecanismo natural para implementar ADTs. Permiten agrupar el estado (atributos) y el comportamiento (métodos) bajo una misma entidad, ocultando la complejidad interna.

Componentes Clave

Estado Privado
Uso de `_variable` para indicar uso interno y proteger datos.

Propiedades (@property)
Acceso de solo lectura a atributos internos sin exponerlos directamente.

Validación en Métodos
Garantizar invariantes (reglas) al modificar el estado.

Representación (__repr__)
Formato de cadena útil para depuración y logging.

Parte 1: Estructura Básica

Ejemplo ADT: Pila (Stack)

Estrategia LIFO

Implementación clásica basada en el principio **Last-In, First-Out**. Utilizamos una lista estándar de Python como estructura de almacenamiento subyacente.

Eficiencia O(1)

- **Push:** append es O(1) amortizado
- **Pop:** eliminar del final es O(1)

Control de Errores

- Validar estado vacío
- Evitar return None silencioso

pila_basica.py

```python
class Pila:
    def __init__(self):
        # Inicializa lista interna vacía
        self._items = []

    def push(self, item):
        # Agrega elemento al tope (final de la lista)
        self._items.append(item)

    def pop(self):
        # Elimina y retorna el elemento del tope
        if self.is_empty():
            raise IndexError("La pila está vacía")
        return self._items.pop()

    def peek(self):
        # Retorna el elemento del tope sin eliminarlo
        if self.is_empty():
            raise IndexError("La pila está vacía")
        return self._items[-1]

    def is_empty(self):
        return not self._items

    def __len__(self):
        return len(self._items)

# --- Ejemplo de Uso ---
p = Pila()
p.push(10)
p.push(20)
print(f"Tope: {p.peek()}")  # Salida: Tope: 20
print(p.pop())           # Salida: 20
```

Estructuras de Datos Robusta

Implementación de Pila (Stack)

stack_generic.py

```python
from typing import Generic, TypeVar, Optional, List

# Variable de tipo para Genéricos
T = TypeVar("T")

class Pila(Generic[T]):
    def __init__(self, max_size: Optional[int] = None):
        self._items: List[T] = []
        self._max = max_size

    def push(self, item: T) -> None:
        # Validación de invariante: capacidad
        if self._max and len(self._items) >= self._max:
            raise OverflowError("Pila llena")
        self._items.append(item)

    def pop(self) -> T:
        if not self._items:
            raise IndexError("Pila vacía")
        return self._items.pop()

    def peek(self) -> T:
        if not self._items:
            raise IndexError("Pila vacía")
        return self._items[-1]

    def __repr__(self) -> str:
        return f"Pila({self._items!r}, max={self._max})"
```

Genéricos (Typing)

`TypeVar`

El uso de `Generic[T]` permite definir una estructura que funciona con cualquier tipo de dato (str, int, obj), manteniendo la seguridad de tipos y ayudando al IDE con el autocompletado.

Validación de Invariantes

`Seguridad`

Se protege la integridad de la estructura. `max_size` previene el consumo infinito de memoria, lanzando excepciones claras (`OverflowError`) antes de que el sistema falle.

Complejidad Temporal

`O(1)`

Las operaciones en el final de una lista de Python (`append` y `pop`) están optimizadas y tienen un costo constante amortizado, ideal para Pilas.

Depuración

`__repr__`

Implementar `__repr__` es crucial para el desarrollo. Permite ver el estado interno del objeto instantáneamente en la consola o logs.

Estructuras de Datos Robusta

Implementación de Pila (Stack)

cola_deque.py

```python
from collections import deque

class Cola:
    def __init__(self):
        # deque optimizado para O(1) en extremos
        self._items = deque()

    def enqueue(self, item):
        # Agrega al final
        self._items.append(item)

    def dequeue(self):
        # Elimina y retorna del inicio (izquierda)
        if self.is_empty():
            raise IndexError("La cola está vacía")
        return self._items.popleft()

    def front(self):
        if self.is_empty():
            raise IndexError("La cola está vacía")
        return self._items[0]

    def is_empty(self):
        return len(self._items) == 0

    def __len__(self):
        return len(self._items)

# --- Ejemplo de uso ---
if __name__ == "__main__":
    c = Cola()
    c.enqueue(1)
    c.enqueue(2)
    print(f"Atendiendo a: {c.dequeue()}")  # 1
    print(f"Siguiente: {c.front()}")       # 2
```

Optimizacion con Deque

Utilizando collections.deque en lugar de una lista estándar. Esto garantiza operaciones de inserción y eliminación en ambos extremos con complejidad O(1).

Rendimiento

- Enqueue: O(1)
- Dequeue: O(1)
- Más rápido que list.pop(0)

¿Por qué no List?

- list.pop(0) es O(n)
- Mueve todos los elementos
- Lento con muchos datos

Estructuras de Datos y Rendimiento

ADT Comunes en Python

ESTRUCTURA	IMPLEMENTACIÓN	CARACTERÍSTICAS Y OPERACIONES	COMPLEJIDAD
Lista	`list`	Secuencia mutable. Indexación rápida, inserción lenta en medio. (Array Dinámico)	`O(1)*`
Pila (Stack)	`list / deque`	**LIFO** (Last-In, First-Out). Solo añadir/quitar del final.	`O(1)`
Cola (Queue)	`deque`	**FIFO** (First-In, First-Out). Enqueue y Dequeue eficientes en extremos.	`O(1)`
Conjunto (Set)	`set`	Colección no ordenada de elementos únicos. Pertenencia muy rápida.	`O(1) avg`
Árbol Básico	`class / dict`	Jerárquico. No nativo. Búsqueda eficiente si está balanceado.	`O(log n)`

Nota sobre Rendimiento

* **O(1) amortizado** significa que la operación es instantánea casi siempre, pero ocasionalmente toma O(n) cuando la lista necesita redimensionarse. Para colas intensivas, prefiere `collections.deque` sobre `list` para evitar O(n) al insertar al inicio.

Guía de Diseño

Mejores Prácticas

Principios clave para desarrollar ADTs robustos y mantenibles en Python.

Elegir Registro
Según el caso de uso

- dict: ad-hoc/rápido
- namedtuple: ligero
- dataclass: modelo

Encapsulamiento
Proteger el estado

- @property
- atributos privados
- interfaces limpias

Tipado & Claridad
Autocompletado y orden

- Type Hints
- Generics [T]
- mypy / pyright

Validación
Asegurar consistencia

- __post_init__
- Guardianes
- Invariantes

Depuración
Facilitar el debug

- __repr__ (dev)
- __str__ (usr)
- __eq__ (tests)

Manejo de Errores
Excepciones claras

- Custom Exceptions
- Mensajes claros
- Fail fast

Testing
Verificar comportamiento

- Unit Tests
- Casos borde
- pytest

Resumen y Conclusiones

Puntos clave para llevar a casa sobre Registros y ADT.

Estructura y Claridad

Los registros agrupan datos relacionados bajo un mismo contexto, eliminando la ambigüedad de variables sueltas y mejorando la legibilidad.

Ecosistema Python

Disponemos de múltiples herramientas nativas para cada necesidad: la flexibilidad de **dict**, la ligereza de **namedtuple** y la potencia de **dataclass**.

Diseño Robusto (ADT)

Separar el **QUÉ** (interfaz) del **CÓMO** (implementación) permite cambiar el código interno sin romper el resto del sistema.

Eficiencia O(1)

Pilas (LIFO) y Colas (FIFO) ofrecen rendimiento constante para sus operaciones principales, resolviendo patrones de acceso específicos de manera óptima.

Validación y Práctica

No existe una "mejor" estructura universal. Analiza los requisitos de mutabilidad, validación y rendimiento para elegir la adecuada.

```python
e = Estudiante(1, "Ana", "ana@uni.es", 8.5)

# Prueba de métodos
print(e.aprobar())  # Output: True
e.actualizar_promedio(9.0)

# Prueba de validación (debe fallar)
try:
    e2 = Estudiante(2, "Luis", "no-email")
except ValueError as err:
    print(err)
```

test_estudiante.py

Ejercicio 1: Estudiante

IMPLEMENTACIÓN CON DATACLASS

CAMPOS Y TIPOS

- **id:** int, **nombre:** str
- **email:** str
- **promedio:** float (defecto: 0.0)
- **activo:** bool (defecto: True)

REQUISITOS LÓGICOS

- Validar: 0.0 ≤ promedio ≤ 10.0
- Validar: email debe contener '@'
- Método: `aprobar() -> bool` (nota ≥ 5)
- Método: `actualizar_promedio(nuevo: float)`

Ejercicio 2:

- Trabajar con **registros (diccionarios)** y una **lista de registros**, realizando operaciones de:
  - Alta
  - Consulta
  - Actualización
  - Eliminación
  - Búsqueda
- Desarrolla un programa en Python que administre un conjunto de empleados utilizando **registros (diccionarios)**.
  - Cada empleado debe tener la siguiente estructura:
- Los registros se almacenarán en una lista llamada empleados.

```
{
    "id": int,
    "nombre": str,
    "edad": int,
    "puesto": str,
    "salario": float,
    "activo": bool
}
```

Ejercicio 3

- Desarrolle un programa en Python que administre productos de una tienda utilizando namedtuple.
- Cada producto debe tener los siguientes campos:
  - id (int)
  - nombre (str)
  - precio (float)
  - stock (int)
  - activo (bool)